

Model Validation Report — IEEE-CIS Fraud Detection

Model validation report — IEEE-CIS fraud detection

Model: `fraud-scoring` (MLflow registry v2) — isotonic-calibrated LightGBM **Status:** **Non-production demonstration** on a public research dataset **Framework:** structured after Federal Reserve **SR 11-7** *Supervisory Guidance on Model Risk Management*. SR 11-7 was superseded on 17 Apr 2026 by **SR 26-2** (*Revised Guidance on Model Risk Management*, which also replaced SR 21-8); SR 26-2 retains the same core framework — conceptual soundness, ongoing monitoring, outcomes analysis, and effective challenge — while emphasizing a **risk-based approach tailored to the institution's model-risk profile**. This report mirrors those concerns, not boilerplate, and cites this repository's real artifacts throughout. **Scope note:** gradient-boosted models such as this one remain **fully in scope** of the revised guidance — SR 26-2 (footnote 3) states its principles “apply to traditional statistical and quantitative models and non-generative, non-agentic AI models,” which is exactly what this is. Generative and agentic AI are placed *outside* the guidance's scope; that exclusion does not touch this conventional supervised classifier.

Data provenance. Computed on the **real Kaggle IEEE-CIS Fraud Detection data** (590,540 labelled transactions; integrity manifest [docs/data_manifest.md](#)). Reproduce end to end with `uv run python -m pipeline.train --full && uv run python -m pipeline.evaluate --report`. This model is a **demonstration** and makes no real fraud decisions.

0. Executive summary

A gradient-boosted fraud model is developed under **leakage-proof temporal validation**, calibrated to produce trustworthy probabilities, and operated through a **dollar-framed** decision threshold. The central finding of the validation is methodological: a random-cross-validation protocol inflates the primary metric by **+17% (XGBoost) to +23% (LightGBM)** over the honest temporal protocol (§4.1) — the reason this model is validated temporally throughout. On the out-of-time holdout the calibrated model achieves **PR-AUC 0.464, ROC-AUC 0.879, Brier 0.023**; at the frozen operating point it captures **47.8% of fraud value at a 4.03% false-positive rate** for **≈ \$111k net value per 100k transactions**. Performance degrades on the out-of-time holdout relative to validation (61% → 48% value capture) — an expected consequence of temporal drift that the monitoring layer (§7) is built to detect. Results are independently reproduced in R (§4.4, effective challenge).

1. Purpose and intended use (SR 11-7 §III — model definition & use)

Purpose. Assign each card transaction a calibrated probability of fraud and, via a cost model, a review/approve decision.

Intended use. *Demonstration only.* This model is **not approved for production use** and makes no real fraud decisions. It exists to demonstrate model-risk-grade development and validation practice on a public dataset. The served demo and API carry the disclaimer “Demonstration system on a public research dataset — not a production fraud decision” ([serving/models.py](#) , the web demo footer).

Development vs usage risk (SR 11-7). *Development risk* is controlled by temporal integrity (§3.2), calibration (§3.4), and independent replication (§4.4). *Usage risk* is bounded by the non-production status, the anonymized-feature interpretation caveat (§6), and the label-lag caveat that accompanies every performance number.

Materiality (SR 26-2). The revised guidance scales the depth of governance to a model’s **materiality** — roughly *exposure × purpose*. By that construct this model is **low-materiality**: it takes no real transactions, moves no money, and makes no customer decision — the served endpoint is explicitly a demonstration. Applying the construct to this document itself, the proportionate response is not lighter validation but validation that *demonstrates the full toolkit at a scale a reviewer can audit* — temporal-integrity tests, calibration, a dollar-framed operating point, and independent replication — which is what follows. A production deployment at real exposure would raise materiality and pull in the controls named as gaps in §6 (fairness assessment, live outcome monitoring).

2. Data (SR 11-7 §V — data quality & relevance)

- **Source.** Kaggle IEEE-CIS Fraud Detection — `train_transaction` + `train_identity` (590,540 labelled transactions; `TransactionDT` is a relative timestamp). The unlabelled competition test set is not used. Integrity (row counts, class balance, SHA-256 checksums) is recorded at fetch time to [docs/data_manifest.md](#) ([pipeline/manifest.py](#)).
- **Class balance.** 20,663 frauds — **3.50%**. Identity is present for 144,233 transactions (**24.4%**) and left-joined.
- **Temporal segmentation** — by `TransactionDT` , documented cut points ([pipeline/data.py](#)):

split	rows	share	fraud rate	role
TRAIN	354,324	60%	3.38%	model fitting
VALIDATION	88,581	15%	4.04%	calibration + threshold + model selection
HOLDOUT	147,635	25%	3.45%	reported once, then replayed for monitoring (§7)

The fraud rate itself varies across the window (3.38% → 4.04% → 3.45%) — real temporal non-stationarity, and part of what §7 monitors.

- **Known dataset limitations.** Features `V1-V339` (Vesta engineered) and `id_12-id_38` are **anonymized** with no published semantics; heavy block-structured missingness; a single fixed observation window (no multi-year seasonality); the competition labels are the ground truth used here.

3. Methodology and assumptions (SR 11-7 §IV — conceptual soundness)

3.1 Features

Base per-transaction transforms (amount log/decimal/round-number flags, time-of-day and day-of-week from `TransactionDT`) plus **causal entity aggregates** (per card / email / address: prior count, prior mean/std amount, recency) ([pipeline/features.py](#)) — 421 numeric + 15 categorical modelling columns.

3.2 Causality — the headline control

Every entity aggregate is computed over **strictly-earlier** transactions only (past-only expanding windows; rows sharing a `TransactionDT` do not see one another). This is enforced by **property-style tests** that shuffle, truncate, and append future rows and assert past-row feature values are byte-for-byte unchanged, plus a brute-force $O(n^2)$ ground-truth check on tie-heavy, NaN-laden data ([tests/test_causality.py](#) , [tests/test_strict_past_bruteforce.py](#)). An adversarial multi-agent audit of this layer found and fixed three temporal-integrity defects during development ([docs/decisions.md](#) D-004).

3.3 Class imbalance

Handled with **cost-sensitive learning** (`scale_pos_weight` / balanced class weights) and threshold optimization (§5). SMOTE and resampling are **deliberately declined** — synthetic minority oversampling distorts the calibration the decision layer depends on and fabricates transactions that never occurred in a temporal stream ([docs/decisions.md](#) D-001).

3.4 Calibration

Probabilities are calibrated with **isotonic regression** (default; Platt compared) fit on VALIDATION only, using a frozen base estimator so the model is never refit ([pipeline/modeling.py](#)). Calibration is **material** here: the cost-weighted base model is badly miscalibrated (Brier 0.052); isotonic corrects it to **0.023** (§4.2). No probability is displayed anywhere unless it comes from the calibrated model. Isotonic carries a known cost — its tied-probability blocks lower ranking-based PR-AUC slightly versus the raw scores — which is measured and weighed against a Platt challenger in §4.2 and §7.

3.5 Assumptions

1. `TransactionDT` ordering is a faithful proxy for real time; (2) blocking a flagged fraud recovers its full `TransactionAmt` ; (3) review cost is a fixed per-alert parameter (default \$25); (4) in this replay, labels are available immediately — **in production they lag weeks** (§6).

4. Performance testing (SR 11-7 §IV — outcomes analysis)

4.1 The leakage experiment — protocol matters more than the model

Same features, same model, two validation protocols ([docs/experiments.md](#)):

model	temporal-val PR-AUC	random-CV PR-AUC	inflation
logistic regression	0.496	0.467	−5.9%
XGBoost	0.606	0.711	+17.3%
LightGBM	0.621	0.765	+23.2%

Random k-fold CV trains on future-dated rows and reports an optimistic PR-AUC. The tree models — the ones actually deployed — are inflated **+17–23%**; the linear model, which does not exploit temporal structure, is not. This is the cost of the anti-pattern, quantified on real data, and the reason every split in this project is temporal.

4.2 Holdout metrics (reported once)

Isotonic-calibrated LightGBM on the HOLDOUT (n=147,635; 5,100 frauds):

metric	value
PR-AUC (primary)	0.464
ROC-AUC	0.879
Brier score	0.0235
recall @ frozen operating point	55.3%

Calibration (fit on VAL, assessed on VAL and HOLDOUT). The uncalibrated cost-weighted model has Brier 0.052; isotonic calibration corrects it to **0.022 (VAL) / 0.023 (HOLDOUT)** — a large, transferable improvement, confirmed by the reliability curve ([docs/figures/reliability.png](#)). This is why calibration is mandatory: the dollar-framed thresholds depend on the probability *level* being right.

Calibration-method tradeoff (effective challenge on the calibrator itself). Isotonic is not free on the ranking metric. On the holdout, isotonic scores PR-AUC **0.464** versus **0.479** for both the raw model and Platt scaling — its step-function output ties large blocks of transactions, and PR-AUC penalises the ambiguous within-tie ordering. Isotonic is nonetheless retained as champion (the a-priori choice, D-005): it delivers the best Brier and the reliability the dollar layer depends on, and the operating point

is chosen on *value*, not raw ranking. The Platt-calibrated LightGBM is registered as the **named challenger** (`fraud-scoring-challenger v1`) so the tradeoff is auditable rather than assumed away (§7; [docs/decisions.md](#) D-009).

4.3 Stability across time (SR 11-7 — performance over time)

The HOLDOUT is replayed as 20 simulated weeks (§7; the `/monitoring/` index). Weekly PR-AUC and value-capture vary across the window and degrade relative to validation (value capture 61% → 48%) — real temporal drift. This instability is the justification for continuous monitoring and recalibration triggers, not a defect to be hidden.

4.4 Independent replication — effective challenge (SR 11-7 §V; SPEC §7b)

The holdout metrics are recomputed **independently in R, with no imports from the Python codebase**, from a single exported scores file (`docs/validation_r/replication.Rmd` reading `holdout_scores.csv`; **148k rows**). R re-derives PR-AUC, ROC-AUC, Brier, the reliability curve, the value-capture curve, and a score-distribution PSI, and **reconciles them against the Python figures within tolerance** (PR-AUC 0.4642, ROC-AUC 0.8788, Brier 0.02347, value capture 0.478 — exact). The R challenge additionally surfaced two subtle **tie-handling** issues in the metric definitions (isotonic produces large tied-probability blocks): a naive per-row R implementation of average precision returns 0.4753, a **0.011 gap that exceeds the 0.01 reconciliation tolerance** on this model, and reconciles only once ties are collapsed at distinct thresholds — exactly the kind of finding an independent implementation is meant to catch (`docs/decisions.md` D-009).

Validator independence (SR 26-2). The revised guidance decouples validation quality from reporting structure — it asks for **rigor and objectivity**, not a particular place on the org chart. This §7b challenge is a *so/o* re-implementation, but it is independent in the way that matters: a separate language and toolchain (R, not Python), no shared code path, working only from an exported scores file, and it materially changed the numbers (the tie-handling fix above). Objectivity is demonstrated by what it caught, not by who signed it.

5. Decision framework (SR 11-7 — model use & interpretation)

The model output is an input to a business decision, framed in dollars (`pipeline/decisions.py`, `docs/decision_report.md`):

- **Cost model.** Net value = (fraud \$ caught by alerts) – review_cost × (alerts). Reviewing costs \$25 (parameter); catching a fraud saves its `TransactionAmt`.
- **Operating point.** Threshold **optimized on VALIDATION, frozen, reported on HOLDOUT** — never tuned on the reported data. At the frozen threshold (**0.122**) the model captures **47.8% of fraud value at 4.03% FPR** (recall 55.3%) for **≈ \$111k net value per 100k transactions**. The tie-block realizability of the operating point was corrected after an adversarial review (`docs/decisions.md`, Phase C).

- **Sensitivity.** The optimal operating point is reported across review costs \$5–\$75 ([docs/decision_report.md](#) §3): higher cost → higher threshold → fewer, higher-precision alerts.
 - **Attribution.** SHAP global importance + fraud/legit/borderline case studies ([pipeline/explain.py](#)), with the anonymized-feature caveat.
-

6. Limitations (SR 11-7 — assumptions & limitations)

- **Anonymized features.** `V*` / `id_*` have no published meaning; SHAP shows *which engineered inputs* move a score, not a mechanistic business reason. Interpretability is structurally capped.
 - **Replay vs production.** Monitoring is a **replayed simulation on a static dataset**, not live traffic; every monitoring page says so.
 - **Label lag.** This replay has labels immediately; in production fraud labels arrive **weeks late**, so live performance metrics are unavailable in real time — drift must be watched through *input* and *score* distributions (§7), not just outcomes.
 - **Dataset vintage.** A single fixed window; no multi-year seasonality or regime change.
 - **No fairness assessment.** The dataset lacks protected-attribute labels; no fairness evaluation is claimed. A production deployment would require one.
-

7. Ongoing monitoring plan (SR 11-7 §IV — ongoing monitoring)

Implemented in [monitoring/](#); published to GitHub Pages [/monitoring/](#) by a daily cron ([.github/workflows/monitoring.yml](#)).

- **What is monitored.** Per simulated week (≈7.4k transactions): Evidently data/prediction drift, explicit **PSI** over the top dense input features and the **score distribution**, and performance on labels (with the label-lag caveat). Causal aggregates are excluded from drift PSI (they grow structurally with history; [docs/decisions.md](#) D-008).
- **Breach thresholds → automated GitHub Issue.** (1) **PSI > 0.2** on any tracked feature; (2) Evidently flags dataset drift; (3) value-capture at the operating point **down > 10%** vs baseline. The alert path is verified by a forced-breach test ([tests/test_monitoring.py](#); `--inject-drift`).
- **Escalation.** A drift Issue is triaged by severity; a sustained value-capture breach or PSI breach on a top-importance input escalates to retraining.
- **Retrain triggers.** (a) sustained value-capture degradation > 10% over two consecutive windows; (b) **PSI > 0.2** on a top-5 importance feature; (c) calibration drift (holdout reliability departing from validation). The recalibration experiment below shows recalibration alone does **not** recover this model's out-of-time loss, so a sustained (a)-type breach escalates to **retraining**, not just a calibration refresh.
- **Recalibration-policy experiment (does recalibration recover the drift?).** A replay experiment ([docs/recalibration_experiment.md](#)) measures the answer instead of asserting it: over the holdout stream the frozen operating point loses ≈ **\$139.6k / 100k** relative to validation, and

periodically **refitting the isotonic map + re-optimizing the threshold** on a trailing labelled window does *not* recover it — recalibrating every 4 or 8 weeks lands **−\$2.1k to −\$2.3k below** frozen, and under a realistic **2-week label lag** it is **−\$9.9k** worse. The degradation is a genuine feature-distribution shift the frozen base model cannot track through calibration alone; this both vindicates freezing the operating point (rather than chasing noisy trailing windows) and confirms retraining as the correct lever for sustained drift. It is a monitoring-policy simulation only — it never changes the frozen headline operating point.

- **Champion / challenger.** The registered model is champion (`fraud-scoring v2`). The **named challenger is the Platt-calibrated LightGBM** registered as `fraud-scoring-challenger v1` — the same base model with a different calibration map, which trades isotonic’s better Brier for higher raw PR-AUC (§4.2). It is scored on the same replay stream; promotion requires beating the champion on temporal-holdout PR-AUC **and** net value at the operating point, never on random-CV.
- **Availability.** A scheduled uptime check pings the demo and `/healthz` ; failure opens a GitHub Issue (`.github/workflows/uptime.yml`) — a dead recruiter-facing link is itself a monitored failure state.

8. Governance & documentation (SR 11-7 §VI)

- **Model inventory.** The model is registered in MLflow (`fraud-scoring v2`, champion; `fraud-scoring-challenger v1`) with its run id; the serving artifact and `/healthz` report the served version + run id (`serving/handler.py`).
- **Reproducibility.** One seeded command rebuilds the model, experiments, and reports; CI runs lint, unit + causality tests, and a seeded smoke-train on every push.
- **Change control.** Infrastructure is Terraform (`infra/`); once applied, ad-hoc console changes are prohibited. `terraform apply` runs only through a controlled, owner-authorized path with CloudWatch billing alarms and API Gateway throttling as cost guards in place first (`docs/decisions.md` D-007, D-012); destructive operations require explicit per-action authorization.
- **Decisions log.** Consequential choices and their rationale are recorded in `docs/decisions.md` .
- **Effective challenge.** Each development phase was subjected to an adversarial, reproduction-driven review; findings and fixes are logged (D-004, Phase C tie-block, Phase D serving hardening, D-009 R reconciliation).

Companion artifacts: [model card](#) · [experiments](#) · [decision report](#) · [data manifest](#) · [R replication](#) · [decisions log](#).